Procesadores del Lenguaje

Curso 2025-2026

Práctica Final – Tercera Parte

Traductor frontend: subconjunto de lenguaje C a LISP (código intermedio)

Traductor backend: LISP a notación Postfija (código final)

En esta parte los objetivos son

- terminar el traductor del frontend
- iniciar la parte de backend

Trabajo a realizar (frontend):

1. Leed el enunciado pasado completo antes de comenzar.
2. Revisad las especificaciones de este documento que tendrá algunas anotaciones añadidas resaltadas. Comprobad que vuestro trabajo ya realizado sigue dichas especificaciones.
3. Renombrad vuestro fichero de código a **trad3.y** .
4. Continuad con el trabajo en el punto que corresponda, procurad guardar una versión de vuestro código cada vez que resolváis un apartado.
5. **Descargad el archivo tests.zip que contiene una serie de programas en C para probar el traductor frontend.**
6. Instalad el intérprete de **`clisp`** de forma local usando

   ```
   sudo apt-get install clisp
   ```

   Probad vuestro programa usando:

   ```
   ./trad3 <prueba.c >prueba.l
   clisp prueba.l
   ```

   o

   ```
   ./trad3 <prueba.c >prueba.l ;  clisp prueba.l
   ```

7. EL INTÉRPRETE WEB NO SE UTILIZARÁ EN LA EVALUACIÓN. Procurad usarlo sólo para pruebas puntuales.

COMPROBAD TAMBIÉN LA PARTE II CORRESPONDIENTE AL BACKEND (siguientes páginas)

Traductor backend: notación LISP a Postfix (código final)

Comenzamos con la segunda parte del traductor que abordará el diseño de un pequeño backend para traducir de Lisp a un lenguaje de bajo nivel para una máquina de pila (como alternativa a las cpus más convencionales programadas con Risc V o lenguaje ensamblador x86) .

Trabajo a realizar (backend):

1. Por favor leed el enunciado completo antes de comenzar a trabajar.
2. Utilizas el fichero back0i.y cómo punto de partida para vuestro trabajo. Renombrad el fichero a **`back3.y`**.esuelva una especificación.
3. Desarrollad los puntos indicados en las Especificaciones.
4. Intentad guardar una versión de su código cada vez que r
5. Descargad el archivo tests.zip que contiene una serie de programas en C para probar vuestros traductores (frontend y backend).
6. Instala el intérprete de **`gforth`** en el portátil siguiendo las instrucciones del apartado *Recursos para realizar la práctica*.
7. Los traductores serán evaluados utilizando el siguiente procedimiento

   `./trad3 <prueba.c | ./back3 | gforth`

   EL INTÉRPRETE WEB NO SE UTILIZARÁ EN LA EVALUACIÓN. Intentad utilizarlo sólo para pruebas específicas.
8. Comprobad que gforth proporcione los mismos resultados que usando:

   `./trad3 <prueba.c >prueba.l; clisp prueba.l`

   O

   `./prueba` después de compilar **`prueba.c`** con **`gcc`**.

Entrega:

Subid el archivo **`back3.y`** con el trabajo realizado hasta ahora junto con el archivo frontend **`trad3.y`**.
Incluid dos líneas de comentarios iniciales en el encabezado del archivo. El primero con vuestros nombres y número de grupo. Y el segundo con los emails (separados por un espacio).
Incluid en el informe **`trad3.pdf`** una breve explicación del trabajo realizado.
Antes y después de realizar la entrega, comprobad que funciona correctamente y que cumple con las indicaciones.

Anexo procedente de la práctica Final - Aproximación 1

Breve Introducción a Forth

Aunque el objetivo de la práctica final es realizar un traductor de C a código intermedio, vamos a comenzar por una fase más elemental: traducir las expresiones que evaluamos con nuestra calculadora en código interpretable en Forth.

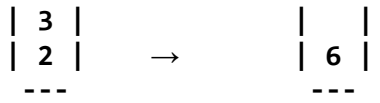
Es decir, es necesario traducir expresiones como

```
2*3<intro>
A=2*3<intro>
B=A*10+23/10<intro>  etc.
```

en un código que sea evaluable por un intérprete de Forth.

Se recomienda recurrir a un intérprete de Forth para ir probando los ejemplos que se plantean aquí. Consultad la última página para instalar **gforth**, o alternativamente al intérprete online: http://nhiro.org/learn_language/FORTH-on-browser.html

La principal particularidad de Forth es que emplea una pila de parámetros sobre la que operan todas las funciones (denominadas *palabras*). Así, la operación de multiplicación * tomará los dos valores de la cima en la pila, para sustituirlos por su producto.



En notación de los así llamados *comentarios de pila*, para la multiplicación tendríamos:

```
(a b – a*b)
```

Para evaluar la expresión **2*3<intro>** en Forth emplearemos la siguiente secuencia:

```
2 3 * . <intro>
```

Los números 2 y 3 se reconocen como tal (*literales*) y son colocados por el intérprete en la pila de parámetros. El operador (*palabra*) * los extrae y los sustituye por el producto de ambos. El operador . es una *palabra* que imprime el valor de la cima de la pila (eliminándolo). Es fácil reconocer que la notación infija a la que estamos habituados (operador binario entre operandos) se ha transformado en notación postfija (polaca inversa: primero los operandos, luego el operador). Quién haya manejado calculadoras de HP estará habituado a esta notación. Adviértase que entre cada uno de los símbolos de entrada debe haber al menos un espacio en blanco. La línea se evalúa al pulsar ***<intro>***, y el intérprete responderá con:

```
6 ok
```

Otros operadores aritméticos son: **+, - y /**. Todos ellos operan de igual forma que la multiplicación. Es importante precisar que los números en la pila son siempre enteros, generalmente con el rango de valores -32768...+32767. La división será por tanto entera y tiene su complemento en el operador `mod` que calcula el resto de una división. Para negar un número existe la *palabra* **negate (a -- -a)**.

Algunas *palabras* habituales de gestión de pila son (entre otras):

- **dup (a -- a a)** duplica el valor de la cima de la pila
- **swap (a b -- b a)** intercambia los dos valores de la cima
- **drop (a b -- a)** elimina el valor de la cima

Cualquiera de las operaciones que se realice sobre una pila con menos operandos de los necesarios provocará un error y, en algunos intérpretes, el vaciado de la pila.

Para definir una variable usaremos la *palabra* **variable**, clasificada en Forth como *palabra constructora*, porque añade una definición al *diccionario* interno. Para crear las variables A y B:

```
variable A  variable B
```

Para realizar la operación **A=6**, usaremos el operador ! .

```
6 A !
```

El intérprete identifica la letra A como una *palabra* definida en el *diccionario* como variable. Así que deja sobre la pila la dirección de memoria asignada a la variable. El operador ! toma de la pila una dirección y un valor, y almacena dicho valor en la dirección. De forma similar podremos resolver **A=3*2** :

```
3 2 * A !
```

Para recuperar el valor de una variable, usaremos el operador **@** :

```
A @ .                          imprimirá:
6 ok
```

La expresión **B=A*10+(23/10)** traducida a Forth debe quedar:.

```
23 10 / A @ 10 * +  B !
B @ .                          imprimirá:
62 ok
```

Aunque no es necesario ahora mismo, podemos dar un pequeño atisbo de la filosofía de este lenguaje. Para construir nuevas *palabras* de tipo función, usaremos otro *constructor*, la *palabra* : .

```
: cuadrado dup * ;
```

Aquí definimos la *palabra* **cuadrado** que contiene la secuencia **dup *** y el terminador de definición **;** . Una aplicación de ésta palabra definida la tenemos en el ejemplo:

```
5 cuadrado .                   que es evaluada con
25 ok
```

En C tendríamos la definición equivalente (no la traducción):

```
int function cuadrado (int valor)
{
      return (valor*valor) ;
}
```

Forth permite el uso de las estructuras de control **`if-then-else`** y **`While`**. **Su uso sólo está permitido dentro de la definición de una palabra (función).**

El **`If-Then-Else`** en versión con la rama **`Else`** es:

```
[condition] IF
    [código a ejecutar si la condición es verdadera]
ELSE
    [opcional: código para ejecutar si la condición es falsa]
THEN
```

- **`[condition]`** es una expresión booleana que debe dejar un valor verdadero (distinto de cero) o falso (cero) en la pila.
- **`IF`** comprueba si el valor superior de la pila es cierto y ejecuta el código entre **`IF`** y **`ELSE`**. Si es falso, la ejecución salta a **`ELSE`** (si está presente) o a **`THEN`**.
- **`ELSE`** es opcional y proporciona una rama alternativa si la condición es falsa.
- **`THEN`** marca el final del bloque **`IF`**.

Ejemplo:

```
: test-condition ( n -- )
    dup 0 >= IF
        ." Numero positivo" cr
    ELSE
        ." Numero no positivo" cr
    THEN ;
```

La versión sin la rama **`ELSE`** es:

```
[condition] IF [código a ejecutar si la condición es verdadera]
THEN
```

La estructura **`WHILE`** se puede implementar con el bucle **`BEGIN ... WHILE ... REPEAT`**

```
BEGIN
    [condition]
WHILE
    [código para repetir]
REPEAT
```

- **`BEGIN`** marca el comienzo del bucle.
- **`[condition]`** es una expresión booleana que debe dejar un valor verdadero (distinto de cero) o falso (cero) en la pila.
- **`WHILE`** comprueba si el valor de la cima de la pila es cierto y ejecuta el código entre **`WHILE`** y **`REPEAT`**. Si es falso, termina después de la etiqueta **`REPEAT`**.
- **`REPEAT`** transfiere la ejecución a la etiqueta **`BEGIN`**.

Ejemplo:

```
: countdown ( n – n )    \ Espera un número en la pila
    BEGIN
        dup 0 >          \ Comprobar si el número es positivo
    WHILE
        dup . cr         \ Imprimir el número
        1 -              \ Decrementa el número
    REPEAT
    drop ;               \ Elimina el cero final de la pila
```

Información adicional:

#9

Usaremos FORTH como una alternativa de CPU / máquina de pila básica con un conjunto de instrucciones limitado. Es por eso que solo se permitirán instrucciones FORTH específicas. En concreto, todas las mencionadas anteriormente:

- Instrucciones de aritmética, lógica y comparación.
- **@** y **!** para manejar variables.
- Los operadores de pila **(drop, dup, swap, over, rot)** seguramente no serán necesarios.
- **:** y **;** para la definición de función
- **IF, THEN, ELSE, DO, WHILE, REPEAT** para estructuras de control.
- **. ." "cr** para impresión y salida de terminal.
- **VARIABLE** para definir variables.
- **CELLS, ALLOT** para vectores no serán necesarios para esta práctica.

No se permiten otras palabras, ni tampoco otros constructores de Forth.

#10

La evaluación de la práctica dependerá de si la concatenación frontend-backend genera los mismos resultados esperados del programa C compilado original o del resultado obtenido en Clisp a partir de la traducción del frontend.

```
./trad <prueba.c | ./back | gforth
./trad <prueba.c >prueba.l; clisp prueba.l
./prueba
```

después de compilar **prueba.c** con **gcc**.

#11

Es muy importante que vuestro backend no imprima al final una llamada a la función principal main.

Esto debe gestionarse exclusivamente mediante la directiva

```
//@ (main)
```

que el frontend traducirá a

```
(main)
```

Y que debe ser traducido por el backend a:

```
main
```

También podría insertarse en el código C la directiva

```
//@  //@  main
```

que el frontend debe transcribir como:

```
//@  main
```

y el backend debe transcribir como:

```
main
```

Para que esto funcione correctamente es imprescindible no imprimir la traducción a nivel de axioma, puesto que en tal caso las llamadas a main aparecerán en primer lugar seguido del código traducido.

Recursos para realizar la práctica:

Utilizaremos gforth para evaluar las prácticas:

Forth

El nivel de conocimiento de Forth necesario para la práctica es elemental, pero dejamos aquí los siguientes enlaces para que sirvan como referencia:

- http://en.wikipedia.org/wiki/Forth_%28programming_language%29 introducción (en inglés)
- http://www.disc.ua.es/~gil/forth.pdf introducción en castellano que más allá de dar una visión del funcionamiento básico, se extiende en los principios de la programación.
- http://www.complang.tuwien.ac.at/forth/gforth/ gforth de GNU.
- http://www.complang.tuwien.ac.at/forth/gforth/Docs-html/ su documentación (en inglés y html)
- http://www.complang.tuwien.ac.at/forth/faq/faq-general.html conjunto de FAQs sobre el lenguaje en sí (en inglés)
- Intérprete gforth online: http://nhiro.org/learn_language/FORTH-on-browser.html

Instrucciones para instalar y comenzar con gforth:

En http://www.complang.tuwien.ac.at/forth/gforth/ están disponibles los fuentes y binarios necesarios para instalar un intérprete de Forth. La instalación en Linux debería ser sencilla siguiendo estos pasos:

- Descargar el fichero **`gforth-0.7.3.tar.gz`** (la versión anterior también debería funcionar, pero da problemas en las aulas informáticas).
- Descomprimir con
  - ~~**`gzip -d gforth-0.7.3.tar.gz`**~~
  - ~~**`tar xvf gforth-0.7.3.tar`**~~
  - **`tar xvf gforth-0.7.3.tar.gz`**
- en el directorio **`gforth-0.7.3`** se ejecuta una configuración con **`./configure`**
- se compila el proyecto con **`make`** y se crean varios intérpretes/compiladores.
- Se puede usar **`gforth`** con el comando **`./gforth`**. Para salir del intérprete, hay que emplear la *palabra* **`bye`**.
- Para trasladar **`gforth`** de directorio hay que copiar también la imagen **`gforth.fi`**.

En Ubuntu podéis usar **`sudo apt-get install gforth`**

En caso de dar error, probad a primero **`sudo apt-get update`**

Si la instalación os da problemas, utilizad guernika para las pruebas y comentad el problema con los profesores.

Especificaciones del BACKEND: Traducción de LISP a FORTH

Utilizad bison para esta parte. Se recomienda abordar cada punto de forma secuencial.

Diseñad la gramática para un kernel LISP básico. Agregad este diseño a la plantilla de bison proporcionada (**back0i.y**). Este archivo contiene, entre otras cosas, varias funciones: yylex, search_keyword, así como una gramática básica y acciones semánticas a modo de referencia que pueden resultar útiles. La gramática incluida evita los conflictos habituales y resuelve algunos de los puntos de la especificación.

Intentad usar traducción directa sin código diferido.

1. Las variables globales se declaran con **`variable <var>`**. Se puede asignar un valor inicial usando **`<valor> <var> !`**
   Traducciones válidas para **`(setq a 1)`** son **`1 variable a a !`** o **`variable a 1 a !`**
   No intentéis crear variables locales o vectores.

2. Las funciones genéricas se definen mediante **`: <fnombre> <code> ;`** y en el caso del procedimiento principal **`: main <code> ;`**
   Vamos a tratar únicamente con el caso del procedimiento principal **`main`**,
   Imprimid siempre espacios antes y después de cada operador de Forth.

3. Para imprimir cadenas, Forth utiliza la palabra **`."`** con otro **`"`** para finalizar la cadena. Ejemplo: **`." Hello World"`**
   El espacio después del **`."`** es importante, pero el **`"`** final no necesita un espacio previo.

4. La impresión de un valor entero se realiza con la palabra **`.`**
   Ejemplos:
   ```
   1 .
   A @ .
   ```

5. Asignar un valor de una expresión a una variable requiere el uso del operador **`!`** (store). Ejemplos:
   ```
   1 A !                    ( A = 1 ; )
   A @  B !                 ( B = A ; )
   A @  B @  * 1 +  B !     ( B = A * B + 1 ; }
   ```

6. La estructura while en Forth es la siguiente:
   `begin [expr] while [code] repeat`
   que es equivalente a **`while (<expr>) { <code> }`**
   dónde **`[expr]`** y **`[code]`** son las traducciones a postfija para **`<expr>`** y **`<code>`**

7. La estructura if es:
   `[expr] if [code1] else [code2] then`
   que es equivalente a **`if <expr> { <code1> } else { <code2> }`**
   Ojo a la sintaxis postfija que se aplica (**`if-else-then`**). En Forth la palabra **`then`** marca el fin del bloque if.
   Para el caso de un if sin rama else: **`[expr] if [code1] then`**

8. La correspondencia entre operadores aritméticos, lógicos y relacionales para los lenguajes C, Lisp y Forth es la siguiente:

C	**&&**	**||**	**!**	**!=**	**==**	**<**	**<=**	**>**	**>=**	**%**
Lisp	**and**	**or**	**not**	**/=**	**=**	**<**	**<=**	**>**	**>=**	**mod**
Forth	**and**	**or**	**0=**	**= 0=**	**=**	**<**	**<=**	**>**	**>=**	**mod**

En C los valores de Falso y Verdadero se suelen representar con 0 y 1. En Forth con 0 y -1.

No se pide ni se evaluará la traducción de Lisp a Forth de vectores, variables locales ni funciones.

Para evaluar el backend se emplearán los seis primeros programas del conjunto de pruebas (carpeta 00).

Para realizar puntos opcionales debéis consultar a los profesores. Sólo se evaluarán en caso de aprobar los exámenes (de evaluación contínua y de convocatoria ordinaria).